Abstracción: para qué sirven las clases

Una guía para leer **antes de la demo 1**. No enseña sintaxis nueva: enseña a **mirar el mundo como lo mira un programador**, para que cuando aparezca la palabra `class` ya sepas por qué existe.

Al terminar deberías poder explicar con tus palabras:

- qué es una **clase** y qué es un **objeto**, y por qué no son lo mismo;
- por qué existe el **encapsulamiento**;
- qué significa "**es un**" (herencia) y "**tiene un**" (composición);
- qué es el **polimorfismo**;
- qué es una **clase abstracta** y qué es una **interfaz**, y cuándo usar cada una.

Cómo se lee esta guía

1. **Primero la historia, después el código.** Cada concepto parte con una analogía de la vida real. El código viene al final y es siempre corto.
2. **Todo el código se ejecuta.** Los ejemplos están en `ejemplos/java/`. Las salidas que ves aquí son las reales, no inventadas.
3. **No hace falta entender cada línea.** Si no conoces algo (`extends`, `private`, `@abstractmethod`), ignóralo en la primera lectura: mira la *forma*, no la sintaxis. La sintaxis la practicas en las demos.
4. **Un hilo conductor.** Los conceptos se explican con **planos y casas** (arquitectura). En la Parte 10 el mismo esqueleto se repite en **11 áreas** distintas: medicina, música, juegos, cocina, fútbol, banco, transporte, mascotas, colegio, redes sociales y arquitectura. Si una no te dice nada, salta a la que sí.
5. **Los ejercicios (Parte 11) no llevan código.** Se hacen con lápiz y papel. Diseñar bien es la mitad del trabajo, y se puede practicar sin teclado.

> Una frase para llevarse: **programar es, sobre todo, decidir qué cosas existen en tu problema y qué sabe hacer cada una.** Escribir la sintaxis viene después.

Parte 1 - El problema: por qué no bastan las variables

Imagina un juego con tres personajes. Cada uno tiene nombre, vida y fuerza de ataque. Con lo que ya sabes (variables y `println`) lo harías así:

```
// SinClases.java
public class SinClases {
    public static void main(String[] args) {
        // Tres personajes de un juego, con variables sueltas
        String nombre1 = "Luna";   int vida1 = 100;   int ataque1 = 12;
        String nombre2 = "Tomas";  int vida2 = 120;   int ataque2 = 9;
        String nombre3 = "Sofia";   int vida3 = 80;    int ataque3 = 15;

        // Luna ataca a Tomas
        vida2 = vida2 - ataque1;
        System.out.println(nombre1 + " ataca a " + nombre2 + ". Vida de " + nombre2
+ ": " + vida2);

        // Sofia ataca a Tomas
        vida2 = vida2 - ataque3;
        System.out.println(nombre3 + " ataca a " + nombre2 + ". Vida de " + nombre2
+ ": " + vida2);

        // Tomas ataca a Luna
        vida1 = vida1 - ataque2;
        System.out.println(nombre2 + " ataca a " + nombre1 + ". Vida de " + nombre1
+ ": " + vida1);
    }
}
```

Salida:

```
Luna ataca a Tomas. Vida de Tomas: 108
Sofia ataca a Tomas. Vida de Tomas: 93
Tomas ataca a Luna. Vida de Luna: 91
```

Funciona. Y aun así, mira lo que pasa si el juego crece:

Si el juego pide...	**Lo que tienes que hacer**
50 personajes en vez de 3	150 variables sueltas con nombres como `vida37`
agregar "mana" a cada personaje	tocar **todo** el programa y crear 50 variables más
que el ataque no pueda dejar la vida bajo 0	copiar la misma validación en cada lugar donde se ataca
un error: `vida2` en vez de `vida3`	el programa compila igual y el bug es invisible

El problema de fondo: **los datos de un personaje andan sueltos y no se sabe cuáles van juntos.** Nada en el código dice "`nombre2`, `vida2` y `ataque2` son la misma persona". Lo sabes tú, de memoria, y eso no escala.

La solución: agrupar datos y comportamiento

Ahora lo mismo, pero describiendo **una sola vez** qué es un héroe:

```
// ConClases.java
class Heroe {
    String nombre;
    int vida;
    int ataque;

    Heroe(String nombre, int vida, int ataque) {
        this.nombre = nombre;
        this.vida = vida;
        this.ataque = ataque;
    }

    void atacar(Heroe objetivo) {
        objetivo.vida = objetivo.vida - ataque;
        System.out.println(nombre + " ataca a " + objetivo.nombre
                + ". Vida de " + objetivo.nombre + ": " + objetivo.vida);
    }
}

public class ConClases {
    public static void main(String[] args) {
        Heroe luna  = new Heroe("Luna", 100, 12);
        Heroe tomas = new Heroe("Tomas", 120, 9);
        Heroe sofia = new Heroe("Sofia", 80, 15);

        luna.atacar(tomas);
        sofia.atacar(tomas);
        tomas.atacar(luna);
    }
}
```

Salida (idéntica a la anterior):

```
Luna ataca a Tomas. Vida de Tomas: 108
Sofia ataca a Tomas. Vida de Tomas: 93
Tomas ataca a Luna. Vida de Luna: 91
```

La salida es la misma, pero el programa cambió de naturaleza:

Sin clases	Con clases
Variables sueltas: `vida1`, `vida2`...	Un héroe **lleva** su `nombre`, `vida` y `ataque`
La lógica de "atacar" se copia y se pega	`atacar` se escribe **una vez** y la tienen todos
Agregar "mana" = tocar todo el programa	Agregar "mana" = una línea en `Heroe`
50 personajes = 150 variables	50 personajes = 50 líneas `new Heroe(...)`

> **Abstraer** es esto: quedarte con lo que importa de algo real (un personaje es: nombre, vida, ataque, y puede atacar) y **olvidar el resto** (su color de pelo, su historia, su voz) hasta que el problema lo pida. Una **clase** es la forma de escribir esa abstracción.

¿Dónde viven las clases en el software de verdad?

En todas partes. Cuando usas una app, lo que ves en pantalla son **objetos** creados a partir de clases:

App	Algunas clases que seguro tiene	Una interfaz probable
Spotify	`Cancion`, `Playlist`, `Usuario`, `Artista`	`Reproducible` (canciones, podcasts, audiolibros)
Instagram	`Perfil`, `Publicacion`, `Comentario`, `Historia`	`Compartible` (fotos, videos, perfiles)
App del banco	`Cuenta`, `Transferencia`, `Tarjeta`, `Cliente`	`MedioDePago` (tarjeta, transferencia, efectivo)
Uber	`Viaje`, `Conductor`, `Pasajero`, `Vehiculo`	`Rastreable` (viajes, vehículos, pedidos)
Netflix	`Pelicula`, `Serie`, `Capitulo`, `Perfil`	`Reproducible`, `Valorable`
Un hospital	`Paciente`, `Medico`, `FichaClinica`, `Examen`	`Cobrable` (consultas, exámenes, camas)

No se inventó la programación orientada a objetos para complicarte la vida: se inventó porque **los programas de millones de líneas no se pueden manejar con variables sueltas.**

Parte 2 - Clase y objeto: el plano y la casa

La analogía. Un arquitecto dibuja el **plano** de una casa: dice cuántos pisos tiene, dónde van las ventanas, cómo se enciende la luz. El plano **no es una casa**: no puedes vivir en él ni encender su luz. Pero con **un** plano se pueden construir **muchas casas**, y cada una tiene su propia dirección y su propia luz encendida o apagada.

Mundo real	Programación	Qué es
El **plano**	La **clase**	La descripción: qué tiene y qué sabe hacer
Cada **casa construida**	Un **objeto** (o *instancia*)	Una cosa concreta, con valores propios
"Tiene dirección, pisos, luz"	**Atributos**	Los datos (lo que el objeto *tiene*)
"Se puede encender la luz"	**Métodos**	Las acciones (lo que el objeto *hace*)
Construir una casa	`new Casa(...)`	Crear un objeto a partir de la clase

```
// PlanoYCasa.java
class Casa {                            // el PLANO
    String direccion;                   // atributos: lo que la casa TIENE
    int pisos;
    boolean luzEncendida;

    Casa(String direccion, int pisos) {
        this.direccion = direccion;
        this.pisos = pisos;
        this.luzEncendida = false;
    }

    void encenderLuz() {                // metodos: lo que la casa HACE
        luzEncendida = true;
        System.out.println("Luz encendida en " + direccion);
    }

    void describir() {
        System.out.println(direccion + ": " + pisos + " piso(s), luz "
                + (luzEncendida ? "encendida" : "apagada"));
    }
}

public class PlanoYCasa {
    public static void main(String[] args) {
        Casa a = new Casa("Los Olmos 123", 2);   // una CASA real (objeto)
        Casa b = new Casa("Las Rosas 45", 1);    // otra casa, del mismo plano

        a.encenderLuz();
        a.describir();
        b.describir();                           // la luz de b sigue apagada
    }
}
```

Salida:

```
Luz encendida en Los Olmos 123
Los Olmos 123: 2 piso(s), luz encendida
Las Rosas 45: 1 piso(s), luz apagada
```

Fíjate en lo importante: `a` y `b` son **dos casas del mismo plano**. Encender la luz de `a` no enciende la de `b`. Cada objeto tiene **su propio estado**.

La misma idea, en 11 áreas

Siempre es lo mismo: una descripción general y muchos casos concretos.

Área	Clase (el plano)	Objeto (el caso real)	Atributos	Métodos
Arquitectura	`Casa`	la casa de Los Olmos 123	dirección, pisos, luz	`encenderLuz()`
Medicina	`Paciente`	Ana, 34 años	nombre, edad, alergias	`pedirHora()`
Música	`Cancion`	"Bohemian Rhapsody"	título, artista, duración	`reproducir()`
Juegos	`Heroe`	Merlina, nivel 12	vida, mana, nivel	`atacar()`
Cocina	`Receta`	la sopa de zapallo de la abuela	ingredientes, tiempo	`preparar()`
Fútbol	`Partido`	Chile vs Argentina	goles, minuto	`anotarGol()`
Banco	`Cliente`	Pedro Pérez	rut, nombre, cuentas	`abrirCuenta()`
Transporte	`Envio`	el envío #1001	origen, destino, estado	`avanzar()`
Mascotas	`CitaVeterinaria`	la cita de Rocky el martes	mascota, fecha, motivo	`reagendar()`
Colegio	`Curso`	Programación, sección 003D	alumnos, profesor	`matricular()`
Redes	`Usuario`	@ana	seguidores, publicaciones	`publicar()`

> **Prueba rápida.** ¿"Perro" es una clase o un objeto? Depende: *el concepto de perro* es una clase; *Rocky, el perro de tu vecino* es un objeto. Si puedes ponerle nombre propio y señalarlo, es un objeto.

Parte 3 - Encapsulamiento: no toques el motor

La analogía. Para acelerar un auto pisas el pedal. No abres el capó a regular la mezcla de combustible. El auto **esconde su mecánica** y te ofrece una **interfaz simple** (pedal, volante, freno).

Gracias a eso, nadie puede dejar el motor en un estado imposible por accidente.

En un programa pasa igual: algunos datos **no deben cambiarse desde afuera como a uno se le ocurra**. Una cuenta no puede tener saldo negativo; una barra de vida no puede pasar de 100; una nota no puede ser 9.5.

```
// Encapsulamiento.java
class Billetera {
    private int saldo;                  // nadie de afuera lo toca directo

    void depositar(int monto) {
        if (monto <= 0) {
            System.out.println("Monto invalido: " + monto);
            return;
        }
        saldo += monto;
    }

    void gastar(int monto) {
        if (monto > saldo) {
            System.out.println("Saldo insuficiente para gastar " + monto);
            return;
        }
        saldo -= monto;
    }

    int getSaldo() {
        return saldo;
    }
}

public class Encapsulamiento {
    public static void main(String[] args) {
        Billetera b = new Billetera();
        b.depositar(5000);
        b.gastar(8000);                 // la clase se defiende sola
        b.depositar(-300);
        // b.saldo = 999999;            // ERROR: saldo es private
        System.out.println("Saldo: " + b.getSaldo());
    }
}
```

Salida:

```
Saldo insuficiente para gastar 8000
Monto invalido: -300
Saldo: 5000
```

La clase `Billetera` **se defiende sola**: aunque alguien llame mal a `gastar`, el saldo nunca queda inválido. La línea comentada `b.saldo = 999999` ni siquiera compila, porque `saldo` es `private`.

La regla de negocio vive dentro de la clase, no regada por el programa. Cada área tiene la suya:

Área	Regla que la clase debe proteger
Medicina	nunca recetar sobre la dosis máxima de un medicamento
Banco	no se puede girar más de lo que hay en la cuenta
Juegos	la vida no baja de 0 ni sube del máximo
Colegio	las notas van de 1.0 a 7.0
Música	el volumen está entre 0 y 100
Transporte	un envío pasa por *pendiente* → *en ruta* → *entregado*, en ese orden

---

Parte 4 - Herencia: "es un"

La analogía. Una *casa* y una *torre de oficinas* son cosas distintas, pero ambas **son un edificio**: tienen dirección, pisos y hay que construirlas. En vez de describir "dirección y pisos" dos veces, lo describes **una vez** en `Edificio`, y `Casa` y `Torre` **heredan** esas características y agregan las suyas.

La prueba para saber si usar herencia es la frase **"es un"**:

- Una casa **es un** edificio. ✓ herencia.
- Un médico **es un** profesional de la salud. ✓ herencia.
- Un auto **es un** motor. ✗ no tiene sentido (eso es otra cosa: Parte 6).

```
// Figuras.java
class Figura {
    String nombre;

    Figura(String nombre) {
        this.nombre = nombre;
    }

    double area() {
        return 0;                       // ¿que area tiene una "figura" a secas?
    }

    void describir() {
        System.out.println(nombre + " tiene area " + area());
    }
}

class Circulo extends Figura {          // un Circulo ES UNA Figura
    double radio;

    Circulo(double radio) {
        super("Circulo");
        this.radio = radio;
    }

    @Override
    double area() {
        return Math.PI * radio * radio;
    }
}

class Rectangulo extends Figura {
    double base, altura;

    Rectangulo(double base, double altura) {
        super("Rectangulo");
        this.base = base;
        this.altura = altura;
    }

    @Override
    double area() {
```

```
        return base * altura;
    }
}

public class Figuras {
    public static void main(String[] args) {
        Figura[] figuras = { new Circulo(1), new Rectangulo(3, 4), new
Figura("Figura generica") };
        for (Figura f : figuras) {
            f.describir();              // misma orden, cada una responde a su
manera
        }
    }
}
```

Salida:

```
Circulo tiene area 3.141592653589793
Rectangulo tiene area 12.0
Figura generica tiene area 0.0
```

`Circulo` y `Rectangulo` **extienden** a `Figura`: reciben `nombre` y `describir()` gratis, y solo escriben lo propio (cómo calcular su área).

¿Y ese `super("Circulo")`?

Fíjate en el constructor de `Circulo`:

```
Circulo(double radio) {
    super("Circulo");        // <- esto
    this.radio = radio;
}
```

`super` significa **"el padre"**, es decir, `Figura`. La línea `super("Circulo")` dice: *"antes de armar lo mío, que `Figura` arme lo suyo y use este nombre"*.

La analogía: al construir una casa, primero se hacen los cimientos y recién después los muros. Un `Circulo` hereda la parte de `Figura` (el `nombre`), y esa parte **tiene que construirse primero**. `super(...)` es la orden de construir los cimientos, y lo hace llamando al constructor del padre.

Por eso:

- **Va siempre como primera línea** del constructor del hijo.

- **Se le pasan los datos que el padre pide.** `Figura(String nombre)` pide un nombre; por eso `Circulo` le manda `"Circulo"`. Así el hijo no repite el código de guardar el nombre: lo hace el padre.
- **Si lo olvidas, no compila**, porque Java no sabe qué nombre ponerle a la parte `Figura`:

```
E4.java:12: error: constructor Figura in class Figura cannot be applied to given
types;
    Circulo(double radio) {
                          ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

(Si el padre **no** tuviera constructor con parámetros, Java pondría el `super()` por ti. Por eso en las primeras demos no lo viste. Apenas el padre pide datos, tú tienes que mandárselos.)

Lo mismo pasa en `Profesional` / `Medico` de la Parte 7: `super(nombre)` le pasa el nombre al padre, que es quien lo guarda.

Pero mira la última línea de la salida: *"Figura generica tiene area 0.0"*. ¿Qué área tiene una "figura" a secas? Ninguna: **no existe una figura que no sea un círculo, un rectángulo o algo concreto.** Esa pregunta sin respuesta es exactamente la razón por la que existen las clases abstractas (Parte 7).

Parte 5 - Polimorfismo: misma orden, distinta respuesta

La analogía. Dices "¡a comer!" en una casa con un perro, un gato y un loro. Todos entienden la orden, pero cada uno la cumple **a su manera**: el perro mueve la cola, el gato maúlla, el loro pide galletas. Tú no necesitas saber *qué animal es* para dar la orden.

En el ejemplo de las figuras ya lo viste, sin que se llamara así:

```
for (Figura f : figuras) {
    f.describir();    // misma orden para todas
}
```

La variable `f` es de tipo `Figura`, pero a veces hay un `Circulo` y a veces un `Rectangulo` detrás. **Java decide en el momento cuál `area()` ejecutar.** Eso es polimorfismo: *"muchas formas"* respondiendo al mismo mensaje.

¿Para qué sirve? Para escribir código que **no cambia** cuando agregas cosas nuevas. Si mañana existe `Triangulo`, ese `for` sigue funcionando sin tocarlo. Sin polimorfismo habría un `if` gigante: *si es círculo..., si es rectángulo..., si es triángulo...* en cada rincón del programa.

Parte 6 - Composición: "tiene un"

La herencia no sirve para todo. Hay una segunda forma de relacionar cosas, y confundirlas es el error más común de quien parte.

La analogía. Un auto **tiene un** motor. No *es* un motor. Si quisieras cambiar el motor por uno eléctrico, no cambiarías el auto entero: cambiarías una pieza.

```
// Composicion.java
class Motor {
    void encender() {
        System.out.println("Motor: brrrum");
    }
}

class Auto {                            // un Auto TIENE UN Motor (no ES un motor)
    private Motor motor = new Motor();

    void partir() {
        System.out.println("Auto: girando la llave");
        motor.encender();
    }
}

public class Composicion {
    public static void main(String[] args) {
        new Auto().partir();
    }
}
```

Salida:

```
Auto: girando la llave
Motor: brrrum
```

`Auto` **guarda** un `Motor` como atributo y le pide cosas. No hereda de él.

Pregunta	Si la respuesta es sí...
¿X **es un** Y?	Herencia (`class X extends Y`)
¿X **tiene un** Y?	Composición (Y es un atributo de X)

Área	"Es un" (herencia)	"Tiene un" (composición)
Arquitectura	una casa **es un** edificio	un edificio **tiene** pisos
Medicina	un médico **es un** profesional	un paciente **tiene** una ficha clínica
Música	una guitarra **es un** instrumento	una banda **tiene** músicos
Juegos	un mago **es un** personaje	un personaje **tiene** un inventario
Cocina	una sopa **es un** plato	una receta **tiene** ingredientes
Fútbol	un portero **es un** jugador	un equipo **tiene** jugadores
Banco	una cuenta corriente **es una** cuenta	un cliente **tiene** cuentas
Transporte	un camión **es un** vehículo	un envío **tiene** paquetes
Mascotas	un perro **es un** animal	un dueño **tiene** mascotas
Colegio	un alumno **es una** persona	un curso **tiene** alumnos
Redes	una foto **es una** publicación	un usuario **tiene** publicaciones

> Cuando dudes entre las dos, **prefiere composición.** Es más flexible, y se rompe menos.

Parte 7 - Clase abstracta: el plano incompleto

Volvamos a la pregunta que quedó abierta en la Parte 4: *¿qué área tiene una figura a secas?* No tiene sentido **crear** una "figura genérica". Pero sí tiene sentido tener `Figura` como **molde común** que obligue a todas las figuras reales a saber calcular su área.

La analogía. Es un **formulario con algunos campos ya llenos y otros en blanco**. Lo llenado (el nombre de la empresa, el logo) es igual para todos. Lo que está en blanco *tiene* que completarlo cada persona, y el formulario no es válido hasta que lo hagan. Nadie puede entregar el formulario "genérico" tal como está.

Una **clase abstracta** es una clase que:

1. **No se puede instanciar**: `new Profesional(...)` es un error.
2. Puede tener **datos y métodos ya hechos** que heredan todos los hijos.
3. Declara **métodos abstractos**: la firma sin cuerpo. Es un "esto lo debes hacer tú, hijo". Cada hijo está **obligado** a escribirlo.

Es la respuesta a *"mis clases comparten cosas Y además quiero forzarlas a completar una parte"*.

El ejemplo completo, con el mundo médico (más abajo hay las otras 10 áreas con el mismo esqueleto):

```
// Medicina.java
abstract class Profesional {
    protected String nombre;

    Profesional(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void atender();                // pendiente: cada hijo lo hace a su
manera
}

interface Cobrable {
    void cobrar();                         // solo la promesa, sin cuerpo
}

class Medico extends Profesional implements Cobrable {
    Medico(String nombre) {
        super(nombre);
    }

    void atender() {
        System.out.println(nombre + " revisa al paciente y receta un tratamiento");
    }

    public void cobrar() {
        System.out.println(nombre + " cobra 25000 por la consulta");
    }
}

class Enfermero extends Profesional {
    Enfermero(String nombre) {
        super(nombre);
    }

    void atender() {
        System.out.println(nombre + " toma la presion y registra los signos
vitales");
```

```
    }
}

// ExamenDeSangre NO es un Profesional, pero tambien cumple el contrato Cobrable
class ExamenDeSangre implements Cobrable {
    private String nombre;

    ExamenDeSangre(String nombre) {
        this.nombre = nombre;
    }

    public void cobrar() {
        System.out.println(nombre + " cuesta 15000 mas el laboratorio");
    }
}

public class Medicina {
    public static void main(String[] args) {
        // new Profesional("x");   // ERROR: Profesional es abstracta, no se puede instanciar

        Profesional[] grupo = { new Medico("Dra. Soto"), new Enfermero("Luis") };
        for (Profesional p : grupo) {
            p.presentar();
            p.atender();
        }

        Cobrable[] lista = { new Medico("Dra. Soto"), new ExamenDeSangre("Examen de sangre") };
        for (Cobrable x : lista) {
            x.cobrar();
        }
    }
}
```

Salida:

```
Soy Dra. Soto
Dra. Soto revisa al paciente y receta un tratamiento
Soy Luis
Luis toma la presion y registra los signos vitales
Dra. Soto cobra 25000 por la consulta
Examen de sangre cuesta 15000 mas el laboratorio
```

Lo que está pasando en `Profesional`:

Línea	Qué significa
`abstract class Profesional`	"Esta clase es un molde, no se crea directamente"
`void presentar() { ... }`	**Ya hecho**: `Medico` y `Enfermero` lo heredan tal cual
`abstract void atender();`	**Pendiente**: sin cuerpo; cada hijo está obligado a escribirlo
`protected String nombre`	Dato común, así no se repite en cada hijo

Qué pasa si lo haces mal (y por qué es bueno que falle)

El error **es la gracia**: la clase abstracta te protege antes de que el programa se ejecute.

1. Intentar crear el molde directamente:

```
Profesional p = new Profesional();
```

```
E1.java:6: error: Profesional is abstract; cannot be instantiated
        Profesional p = new Profesional();
                        ^
```

2. Un hijo que olvida completar lo pendiente:

```
class Medico extends Profesional {
    // (falta atender())
}
```

```
E3.java:4: error: Medico is not abstract and does not override abstract method
atender() in Profesional
class Medico extends Profesional {
^
```

Parte 8 - Interfaz: el contrato, la habilidad

La clase abstracta responde a "**qué es**". La interfaz responde a una pregunta distinta: "**qué sabe hacer**".

La analogía. Un **enchufe**. A un enchufe le da lo mismo si le conectas una lámpara, un taladro o un cargador: lo único que importa es que el aparato tenga la **clavija correcta**. Lámpara y taladro no se parecen en nada (no son "de la misma familia"), pero ambos **cumplen el contrato** del enchufe.

Una **interfaz**:

1. Es **solo una lista de promesas**: nombres de métodos sin cuerpo.
2. Cualquier clase puede **firmar** el contrato (`implements`), venga de la familia que venga.
3. Una clase puede firmar **varios** contratos a la vez.

Por eso la interfaz cruza familias. En el ejemplo médico, `Cobrable` lo cumplen un `Medico` (que es un profesional) y un `ExamenDeSangre` (que **no** es un profesional, no tiene nada que ver con ellos). Lo único que tienen en común es que **se pueden cobrar**:

```
interface Cobrable {
    void cobrar();            // solo la promesa, sin cuerpo
}
```

Y quien lo use no necesita saber qué es cada cosa:

```
Cobrable[] lista = { new Medico("Dra. Soto"), new ExamenDeSangre("Examen de sangre")
};
for (Cobrable x : lista) {
    x.cobrar();               // le da igual si es un medico o un examen
}
```

(Está en `Medicina.java`, la misma salida que viste en la Parte 7: las dos últimas líneas.)

El error equivalente: firmar un contrato y no cumplirlo

```
class Medico implements Cobrable {
    // (falta cobrar())
}
```

```
E2.java:4: error: Medico is not abstract and does not override abstract method
cobrar() in Cobrable
class Medico implements Cobrable {
^
```

Recuerda: en Java una clase puede tener **un** solo `extends`, pero **varios** `implements`.

Parte 9 - Abstracta vs. interfaz: cómo elegir

Es **la** confusión del curso. Esta tabla es para volver a ella:

	Clase abstracta	Interfaz
Responde a...	"**Qué es**" (es un)	"**Qué sabe hacer**" (habilidad)
Analogía	formulario a medio llenar	enchufe / contrato
¿Se puede hacer `new`?	No	No
¿Tiene datos (atributos)?	**Sí**	No (en lo básico)
¿Tiene métodos ya hechos?	**Sí**, y también abstractos	Solo promesas (en lo básico)
¿Cuántas puede usar un hijo?	**Una** sola (`extends`)	**Todas las que quiera** (`implements`)
¿Une clases de familias distintas?	No: los hijos son "parientes"	**Sí**: pueden no tener relación
Palabra en Java	`abstract class`	`interface`

Árbol de decisión

```
¿Voy a crear objetos directamente de esto?
│
├── SI  → clase normal.
│
└── NO  ¿Los hijos comparten datos o código ya hecho?
        │
        ├── SI  → clase abstracta   ("es un": un Medico ES UN Profesional)
        │
        └── NO  ¿Solo quiero exigir que sepan hacer algo,
                 sin importar qué son?
                 │
                 └── SI  → interfaz  ("sabe hacer": Cobrable, Volador, Compartible)
```

Y las dos juntas son lo más habitual: el `Medico` **es un** `Profesional` (abstracta) y **sabe** `cobrar` (interfaz).

Tres frases para recordar

- **"Es un"** → herencia (clase normal o abstracta).
- **"Tiene un"** → composición (un atributo).
- **"Sabe hacer"** → interfaz.

Un ejemplo para decidir: los pingüinos

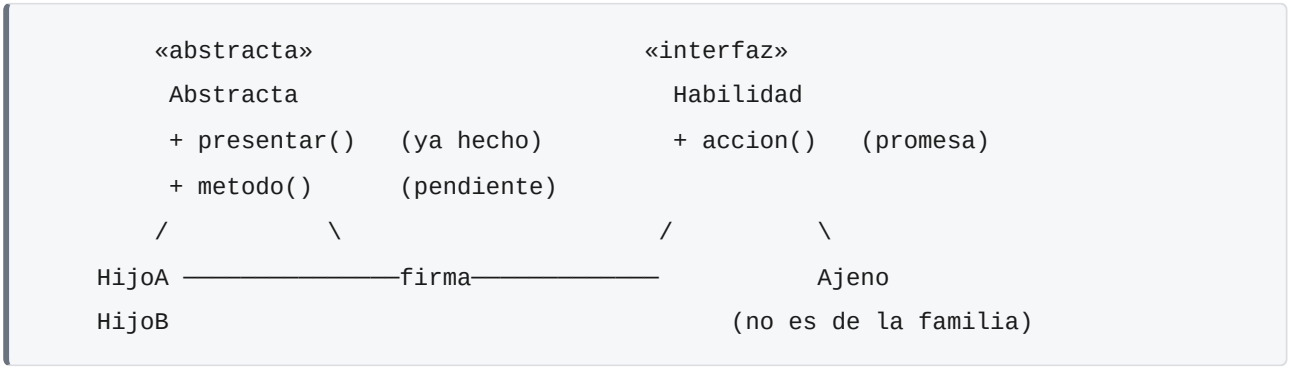
Alguien diseña `Ave` con un método `volar()`. Luego crea `Gorrion` y `Pinguino`. Problema: **el pingüino es un ave que no vuela.** Si `volar()` está en `Ave`, el pingüino hereda algo falso.

La salida: `Ave` solo tiene lo que **todas** las aves tienen (`ponerHuevos()`), y `volar()` pasa a una interfaz `Volador` que **solo firman las que vuelan**. Un `Dragon` o un `Avion` también pueden firmarla, aunque no sean aves: eso es justamente que la interfaz cruza familias.

(El ejemplo de juegos de la Parte 10 hace exactamente esto con un mago y un dragón.)

Parte 10 - El mismo esqueleto, en 11 áreas

Todas estas áreas se resuelven con **la misma estructura**. Mira la forma, no el tema:



En cada ficha: **qué clases hay**, **qué se protege con encapsulamiento**, quién **es un** y quién **tiene un**. Después, el código y la salida real. Todos los archivos están en `ejemplos/java/`.

10.1 Medicina

Un hospital está lleno de gente distinta (médicos, enfermeros) y de cosas que se cobran (consultas, exámenes). Dos relaciones distintas: *quién es qué* y *qué se puede cobrar*.

Clase (plano)	`Paciente` -> objeto: Ana, 34 años
Encapsulamiento	la dosis de un medicamento nunca supera el máximo
Abstracta (es un)	`Profesional` -> `Medico`, `Enfermero`

Interfaz (sabe hacer)	`Cobrable` -> `Medico` **y** `ExamenDeSangre` (que no es profesional)
Composición (tiene un)	un `Paciente` tiene una `FichaClinica`

```
// Medicina.java
abstract class Profesional {
    protected String nombre;

    Profesional(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void atender();                // pendiente: cada hijo lo hace a su
manera
}

interface Cobrable {
    void cobrar();                         // solo la promesa, sin cuerpo
}

class Medico extends Profesional implements Cobrable {
    Medico(String nombre) {
        super(nombre);
    }

    void atender() {
        System.out.println(nombre + " revisa al paciente y receta un tratamiento");
    }

    public void cobrar() {
        System.out.println(nombre + " cobra 25000 por la consulta");
    }
}

class Enfermero extends Profesional {
    Enfermero(String nombre) {
        super(nombre);
    }

    void atender() {
        System.out.println(nombre + " toma la presion y registra los signos
vitales");
```

```
    }
}

// ExamenDeSangre NO es un Profesional, pero tambien cumple el contrato Cobrable
class ExamenDeSangre implements Cobrable {
    private String nombre;

    ExamenDeSangre(String nombre) {
        this.nombre = nombre;
    }

    public void cobrar() {
        System.out.println(nombre + " cuesta 15000 mas el laboratorio");
    }
}

public class Medicina {
    public static void main(String[] args) {
        // new Profesional("x");   // ERROR: Profesional es abstracta, no se puede
instanciar

        Profesional[] grupo = { new Medico("Dra. Soto"), new Enfermero("Luis") };
        for (Profesional p : grupo) {
            p.presentar();
            p.atender();
        }

        Cobrable[] lista = { new Medico("Dra. Soto"), new ExamenDeSangre("Examen de
sangre") };
        for (Cobrable x : lista) {
            x.cobrar();
        }
    }
}
```

Salida:

```
Soy Dra. Soto
Dra. Soto revisa al paciente y receta un tratamiento
Soy Luis
Luis toma la presion y registra los signos vitales
Dra. Soto cobra 25000 por la consulta
Examen de sangre cuesta 15000 mas el laboratorio
```

10.2 Musica

Una guitarra y un piano son instrumentos, pero también hay cosas que *no* son instrumentos y se conectan a un amplificador: un micrófono.

Clase (plano)	`Cancion` -> objeto: "Bohemian Rhapsody"
Encapsulamiento	el volumen se mantiene entre 0 y 100
Abstracta (es un)	`Instrumento` -> `Guitarra`, `Piano`
Interfaz (sabe hacer)	`Amplificable` -> `Guitarra` **y** `Microfono`
Composición (tiene un)	una `Banda` tiene `Musicos`; un `Album` tiene `Canciones`

```
// Musica.java
abstract class Instrumento {
    protected String nombre;

    Instrumento(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void tocar();                // pendiente: cada hijo lo hace a su
manera
}

interface Amplificable {
    void amplificar();                        // solo la promesa, sin cuerpo
}

class Guitarra extends Instrumento implements Amplificable {
    Guitarra(String nombre) {
        super(nombre);
    }

    void tocar() {
        System.out.println(nombre + " rasguea un acorde de mi menor");
    }

    public void amplificar() {
        System.out.println(nombre + " se conecta al amplificador y suena fuerte");
    }
}

class Piano extends Instrumento {
    Piano(String nombre) {
        super(nombre);
    }

    void tocar() {
        System.out.println(nombre + " toca una escala de do mayor");
    }
```

```
}

// Microfono NO es un Instrumento, pero tambien cumple el contrato Amplificable
class Microfono implements Amplificable {
    private String nombre;

    Microfono(String nombre) {
        this.nombre = nombre;
    }

    public void amplificar() {
        System.out.println(nombre + " capta la voz y la manda al parlante");
    }
}

public class Musica {
    public static void main(String[] args) {
        // new Instrumento("x");   // ERROR: Instrumento es abstracta, no se puede
instanciar

        Instrumento[] grupo = { new Guitarra("Guitarra electrica"), new Piano("Piano
de cola") };
        for (Instrumento p : grupo) {
            p.presentar();
            p.tocar();
        }

        Amplificable[] lista = { new Guitarra("Guitarra electrica"), new
Microfono("Microfono") };
        for (Amplificable x : lista) {
            x.amplificar();
        }
    }
}
```

Salida:

```
Soy Guitarra electrica
Guitarra electrica rasguea un acorde de mi menor
Soy Piano de cola
Piano de cola toca una escala de do mayor
Guitarra electrica se conecta al amplificador y suena fuerte
Microfono capta la voz y la manda al parlante
```

10.3 Juegos

Es el caso de los pingüinos de la Parte 9: el dragón vuela y no es un personaje; el guerrero es un personaje y no vuela.

Clase (plano)	`Heroe` -> objeto: Merlina, nivel 12
Encapsulamiento	la vida nunca baja de 0 ni supera el máximo
Abstracta (es un)	`Personaje` -> `Mago`, `Guerrero`
Interfaz (sabe hacer)	`Volador` -> `Mago` **y** `Dragon` (monstruo, no personaje)
Composición (tiene un)	un `Personaje` tiene un `Inventario`

```
// Juegos.java
abstract class Personaje {
    protected String nombre;

    Personaje(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void atacar();                 // pendiente: cada hijo lo hace a su
manera
}

interface Volador {
    void volar();                         // solo la promesa, sin cuerpo
}

class Mago extends Personaje implements Volador {
    Mago(String nombre) {
        super(nombre);
    }

    void atacar() {
        System.out.println(nombre + " lanza una bola de fuego");
    }

    public void volar() {
        System.out.println(nombre + " vuela sobre el campo de batalla");
    }
}

class Guerrero extends Personaje {
    Guerrero(String nombre) {
        super(nombre);
    }

    void atacar() {
        System.out.println(nombre + " golpea con su espada");
    }
```

```
}

// Dragon NO es un Personaje, pero tambien cumple el contrato Volador
class Dragon implements Volador {
    private String nombre;

    Dragon(String nombre) {
        this.nombre = nombre;
    }

    public void volar() {
        System.out.println(nombre + " vuela y escupe fuego");
    }
}

public class Juegos {
    public static void main(String[] args) {
        // new Personaje("x");   // ERROR: Personaje es abstracta, no se puede
instanciar

        Personaje[] grupo = { new Mago("Merlina"), new Guerrero("Thor") };
        for (Personaje p : grupo) {
            p.presentar();
            p.atacar();
        }

        Volador[] lista = { new Mago("Merlina"), new Dragon("Smaug") };
        for (Volador x : lista) {
            x.volar();
        }
    }
}
```

Salida:

```
Soy Merlina
Merlina lanza una bola de fuego
Soy Thor
Thor golpea con su espada
Merlina vuela sobre el campo de batalla
Smaug vuela y escupe fuego
```

10.4 Cocina

Un plato es una cosa; *poder calentarse* es otra. Una sopa y un café se calientan, pero la ensalada no.

Clase (plano)	`Receta` -> objeto: la sopa de zapallo de la abuela
Encapsulamiento	la temperatura del horno tiene un máximo
Abstracta (es un)	`Plato` -> `Sopa`, `Ensalada`
Interfaz (sabe hacer)	`Calentable` -> `Sopa` **y** `Cafe` (una bebida)
Composición (tiene un)	una `Receta` tiene `Ingredientes`

```
// Cocina.java
abstract class Plato {
    protected String nombre;

    Plato(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void preparar();                // pendiente: cada hijo lo hace a su
manera
}

interface Calentable {
    void calentar();                        // solo la promesa, sin cuerpo
}

class Sopa extends Plato implements Calentable {
    Sopa(String nombre) {
        super(nombre);
    }

    void preparar() {
        System.out.println(nombre + " hierve el zapallo y lo muele");
    }

    public void calentar() {
        System.out.println(nombre + " se calienta a fuego lento");
    }
}

class Ensalada extends Plato {
    Ensalada(String nombre) {
        super(nombre);
    }

    void preparar() {
        System.out.println(nombre + " corta tomate y cebolla");
    }
```

```
}

// Cafe NO es un Plato, pero tambien cumple el contrato Calentable
class Cafe implements Calentable {
    private String nombre;

    Cafe(String nombre) {
        this.nombre = nombre;
    }

    public void calentar() {
        System.out.println(nombre + " se calienta en la maquina");
    }
}

public class Cocina {
    public static void main(String[] args) {
        // new Plato("x");   // ERROR: Plato es abstracta, no se puede instanciar

        Plato[] grupo = { new Sopa("Sopa de zapallo"), new Ensalada("Ensalada
chilena") };
        for (Plato p : grupo) {
            p.presentar();
            p.preparar();
        }

        Calentable[] lista = { new Sopa("Sopa de zapallo"), new Cafe("Cafe de
grano") };
        for (Calentable x : lista) {
            x.calentar();
        }
    }
}
```

Salida:

```
Soy Sopa de zapallo
Sopa de zapallo hierve el zapallo y lo muele
Soy Ensalada chilena
Ensalada chilena corta tomate y cebolla
Sopa de zapallo se calienta a fuego lento
Cafe de grano se calienta en la maquina
```

10.5 Fútbol

Delanteros y porteros son jugadores. Pero la tarjeta roja también se la puede mostrar a un entrenador, que no juega.

Clase (plano)	`Partido` -> objeto: Chile vs Argentina
Encapsulamiento	el marcador solo cambia con `anotarGol()`, nunca a mano
Abstracta (es un)	`Jugador` -> `Delantero`, `Portero`
Interfaz (sabe hacer)	`Sancionable` -> `Delantero` **y** `Entrenador`
Composición (tiene un)	un `Equipo` tiene `Jugadores`

```
// Futbol.java
abstract class Jugador {
    protected String nombre;

    Jugador(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void jugar();                // pendiente: cada hijo lo hace a su
manera
}

interface Sancionable {
    void sancionar();                        // solo la promesa, sin cuerpo
}

class Delantero extends Jugador implements Sancionable {
    Delantero(String nombre) {
        super(nombre);
    }

    void jugar() {
        System.out.println(nombre + " remata al arco");
    }

    public void sancionar() {
        System.out.println(nombre + " recibe tarjeta amarilla");
    }
}

class Portero extends Jugador {
    Portero(String nombre) {
        super(nombre);
    }

    void jugar() {
        System.out.println(nombre + " ataja un penal");
    }
```

```
}

// Entrenador NO es un Jugador, pero tambien cumple el contrato Sancionable
class Entrenador implements Sancionable {
    private String nombre;

    Entrenador(String nombre) {
        this.nombre = nombre;
    }

    public void sancionar() {
        System.out.println(nombre + " recibe tarjeta roja desde el banco");
    }
}

public class Futbol {
    public static void main(String[] args) {
        // new Jugador("x");   // ERROR: Jugador es abstracta, no se puede
instanciar

        Jugador[] grupo = { new Delantero("Alexis"), new Portero("Bravo") };
        for (Jugador p : grupo) {
            p.presentar();
            p.jugar();
        }

        Sancionable[] lista = { new Delantero("Alexis"), new Entrenador("Bielsa") };
        for (Sancionable x : lista) {
            x.sancionar();
        }
    }
}
```

Salida:

```
Soy Alexis
Alexis remata al arco
Soy Bravo
Bravo ataja un penal
Alexis recibe tarjeta amarilla
Bielsa recibe tarjeta roja desde el banco
```

10.6 Banco

Las cuentas se parecen entre sí. Pero lo que audita el banco también incluye transferencias, que no son cuentas.

Clase (plano)	`Cliente` -> objeto: Pedro Pérez
Encapsulamiento	el saldo es privado: solo cambia con `depositar` y `girar`
Abstracta (es un)	`Cuenta` -> `CuentaCorriente`, `CuentaAhorro`
Interfaz (sabe hacer)	`Auditable` -> `CuentaCorriente` **y** `Transferencia`
Composición (tiene un)	un `Cliente` tiene `Cuentas`

```
// Banco.java
abstract class Cuenta {
    protected String nombre;

    Cuenta(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void cobrarMantencion();                // pendiente: cada hijo lo
hace a su manera
}

interface Auditable {
    void auditar();                         // solo la promesa, sin cuerpo
}

class CuentaCorriente extends Cuenta implements Auditable {
    CuentaCorriente(String nombre) {
        super(nombre);
    }

    void cobrarMantencion() {
        System.out.println(nombre + " cobra 3000 de mantencion");
    }

    public void auditar() {
        System.out.println(nombre + " entrega su historial al auditor");
    }
}

class CuentaAhorro extends Cuenta {
    CuentaAhorro(String nombre) {
        super(nombre);
    }

    void cobrarMantencion() {
        System.out.println(nombre + " no cobra mantencion");
    }
```

```
}

// Transferencia NO es un Cuenta, pero tambien cumple el contrato Auditable
class Transferencia implements Auditable {
    private String nombre;

    Transferencia(String nombre) {
        this.nombre = nombre;
    }

    public void auditar() {
        System.out.println(nombre + " entrega su registro al auditor");
    }
}

public class Banco {
    public static void main(String[] args) {
        // new Cuenta("x");   // ERROR: Cuenta es abstracta, no se puede instanciar

        Cuenta[] grupo = { new CuentaCorriente("Cuenta corriente 001"), new
CuentaAhorro("Cuenta de ahorro 002") };
        for (Cuenta p : grupo) {
            p.presentar();
            p.cobrarMantencion();
        }

        Auditable[] lista = { new CuentaCorriente("Cuenta corriente 001"), new
Transferencia("Transferencia 9001") };
        for (Auditable x : lista) {
            x.auditar();
        }
    }
}
```

Salida:

```
Soy Cuenta corriente 001
Cuenta corriente 001 cobra 3000 de mantencion
Soy Cuenta de ahorro 002
Cuenta de ahorro 002 no cobra mantencion
Cuenta corriente 001 entrega su historial al auditor
Transferencia 9001 entrega su registro al auditor
```

10.7 Transporte

Es el mundo de **DUOC Delivery** (el taller grupal). Un camión y una moto son vehículos; el camión y el paquete se pueden rastrear.

Clase (plano)	`Envio` -> objeto: el envío #1001
Encapsulamiento	un envío pasa por *pendiente, en ruta, entregado*, en ese orden
Abstracta (es un)	`Vehiculo` -> `Camion`, `Moto`
Interfaz (sabe hacer)	`Rastreable` -> `Camion` **y** `Paquete`
Composición (tiene un)	un `Envio` tiene `Paquetes`; una `Ruta` tiene `Paradas`

```
// Transporte.java
abstract class Vehiculo {
    protected String nombre;

    Vehiculo(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void entregar();                // pendiente: cada hijo lo hace a su
manera
}

interface Rastreable {
    void ubicar();                         // solo la promesa, sin cuerpo
}

class Camion extends Vehiculo implements Rastreable {
    Camion(String nombre) {
        super(nombre);
    }

    void entregar() {
        System.out.println(nombre + " lleva 20 cajas por carretera");
    }

    public void ubicar() {
        System.out.println(nombre + " reporta su GPS: ruta 5 sur");
    }
}

class Moto extends Vehiculo {
    Moto(String nombre) {
        super(nombre);
    }

    void entregar() {
        System.out.println(nombre + " entrega un sobre en 10 minutos");
    }
```

```
}

// Paquete NO es un Vehiculo, pero tambien cumple el contrato Rastreable
class Paquete implements Rastreable {
    private String nombre;

    Paquete(String nombre) {
        this.nombre = nombre;
    }

    public void ubicar() {
        System.out.println(nombre + " reporta su GPS: bodega central");
    }
}

public class Transporte {
    public static void main(String[] args) {
        // new Vehiculo("x");   // ERROR: Vehiculo es abstracta, no se puede
instanciar

        Vehiculo[] grupo = { new Camion("Camion 7"), new Moto("Moto 3") };
        for (Vehiculo p : grupo) {
            p.presentar();
            p.entregar();
        }

        Rastreable[] lista = { new Camion("Camion 7"), new Paquete("Paquete 55") };
        for (Rastreable x : lista) {
            x.ubicar();
        }
    }
}
```

Salida:

```
Soy Camion 7
Camion 7 lleva 20 cajas por carretera
Soy Moto 3
Moto 3 entrega un sobre en 10 minutos
Camion 7 reporta su GPS: ruta 5 sur
Paquete 55 reporta su GPS: bodega central
```

10.8 Mascotas y veterinaria

Perros y gatos son animales. Pero "se le puede enseñar trucos" lo puede cumplir hasta un robot mascota, que no es un animal.

Clase (plano)	`CitaVeterinaria` -> objeto: la cita de Rocky el martes
Encapsulamiento	el peso de una mascota nunca puede ser negativo
Abstracta (es un)	`Animal` -> `Perro`, `Gato`
Interfaz (sabe hacer)	`Adiestrable` -> `Perro` **y** `RoboMascota`
Composición (tiene un)	un `Dueno` tiene `Mascotas`

```
// Mascotas.java
abstract class Animal {
    protected String nombre;

    Animal(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void hacerSonido();                // pendiente: cada hijo lo hace a
su manera
}

interface Adiestrable {
    void adiestrar();                       // solo la promesa, sin cuerpo
}

class Perro extends Animal implements Adiestrable {
    Perro(String nombre) {
        super(nombre);
    }

    void hacerSonido() {
        System.out.println(nombre + " dice guau");
    }

    public void adiestrar() {
        System.out.println(nombre + " aprende a sentarse");
    }
}

class Gato extends Animal {
    Gato(String nombre) {
        super(nombre);
    }

    void hacerSonido() {
        System.out.println(nombre + " dice miau");
    }
```

```
}

// RoboMascota NO es un Animal, pero tambien cumple el contrato Adiestrable
class RoboMascota implements Adiestrable {
    private String nombre;

    RoboMascota(String nombre) {
        this.nombre = nombre;
    }

    public void adiestrar() {
        System.out.println(nombre + " aprende a dar la pata");
    }
}

public class Mascotas {
    public static void main(String[] args) {
        // new Animal("x");   // ERROR: Animal es abstracta, no se puede instanciar

        Animal[] grupo = { new Perro("Rocky"), new Gato("Michi") };
        for (Animal p : grupo) {
            p.presentar();
            p.hacerSonido();
        }

        Adiestrable[] lista = { new Perro("Rocky"), new RoboMascota("Robo-Fido") };
        for (Adiestrable x : lista) {
            x.adiestrar();
        }
    }
}
```

Salida:

```
Soy Rocky
Rocky dice guau
Soy Michi
Michi dice miau
Rocky aprende a sentarse
Robo-Fido aprende a dar la pata
```

10.9 Colegio

Alumnos y profesores son personas. Pero *recibir una nota* también le pasa a un proyecto, que no es persona.

Clase (plano)	`Curso` -> objeto: Programación, sección 003D
Encapsulamiento	las notas están entre 1.0 y 7.0
Abstracta (es un)	`Persona` -> `Alumno`, `Profesor`
Interfaz (sabe hacer)	`Evaluable` -> `Alumno` **y** `Proyecto`
Composición (tiene un)	un `Curso` tiene `Alumnos`

```
// Colegio.java
abstract class Persona {
    protected String nombre;

    Persona(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void participar();                 // pendiente: cada hijo lo hace a su
manera
}

interface Evaluable {
    void evaluar();                         // solo la promesa, sin cuerpo
}

class Alumno extends Persona implements Evaluable {
    Alumno(String nombre) {
        super(nombre);
    }

    void participar() {
        System.out.println(nombre + " levanta la mano y responde");
    }

    public void evaluar() {
        System.out.println(nombre + " recibe nota 6.5");
    }
}

class Profesor extends Persona {
    Profesor(String nombre) {
        super(nombre);
    }

    void participar() {
        System.out.println(nombre + " explica el contenido en la pizarra");
    }
```

```
}

// Proyecto NO es un Persona, pero tambien cumple el contrato Evaluable
class Proyecto implements Evaluable {
    private String nombre;

    Proyecto(String nombre) {
        this.nombre = nombre;
    }

    public void evaluar() {
        System.out.println(nombre + " recibe nota 6.0");
    }
}

public class Colegio {
    public static void main(String[] args) {
        // new Persona("x");   // ERROR: Persona es abstracta, no se puede
instanciar

        Persona[] grupo = { new Alumno("Camila"), new Profesor("Sr. Rojas") };
        for (Persona p : grupo) {
            p.presentar();
            p.participar();
        }

        Evaluable[] lista = { new Alumno("Camila"), new Proyecto("Proyecto de
ciencias") };
        for (Evaluable x : lista) {
            x.evaluar();
        }
    }
}
```

Salida:

```
Soy Camila
Camila levanta la mano y responde
Soy Sr. Rojas
Sr. Rojas explica el contenido en la pizarra
Camila recibe nota 6.5
Proyecto de ciencias recibe nota 6.0
```

10.10 Redes sociales y streaming

Fotos y videos son publicaciones. Pero hasta un perfil se puede compartir, y un perfil no es una publicación.

Clase (plano)	`Usuario` -> objeto: @ana
Encapsulamiento	la contraseña es privada y **no** tiene getter
Abstracta (es un)	`Publicacion` -> `Foto`, `Video`
Interfaz (sabe hacer)	`Compartible` -> `Foto` **y** `Perfil`
Composición (tiene un)	un `Usuario` tiene `Publicaciones`; una `Playlist` tiene `Canciones`

```
// Redes.java
abstract class Publicacion {
    protected String nombre;

    Publicacion(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void mostrar();                // pendiente: cada hijo lo hace a su
manera
}

interface Compartible {
    void compartir();                        // solo la promesa, sin cuerpo
}

class Foto extends Publicacion implements Compartible {
    Foto(String nombre) {
        super(nombre);
    }

    void mostrar() {
        System.out.println(nombre + " se muestra en pantalla completa");
    }

    public void compartir() {
        System.out.println(nombre + " se comparte con 120 personas");
    }
}

class Video extends Publicacion {
    Video(String nombre) {
        super(nombre);
    }

    void mostrar() {
        System.out.println(nombre + " se reproduce en bucle");
    }
```

```
}

// Perfil NO es un Publicacion, pero tambien cumple el contrato Compartible
class Perfil implements Compartible {
    private String nombre;

    Perfil(String nombre) {
        this.nombre = nombre;
    }

    public void compartir() {
        System.out.println(nombre + " se comparte con 300 seguidores");
    }
}

public class Redes {
    public static void main(String[] args) {
        // new Publicacion("x");   // ERROR: Publicacion es abstracta, no se puede
instanciar

        Publicacion[] grupo = { new Foto("Foto del atardecer"), new Video("Video de
la fiesta") };
        for (Publicacion p : grupo) {
            p.presentar();
            p.mostrar();
        }

        Compartible[] lista = { new Foto("Foto del atardecer"), new Perfil("Perfil
de Ana") };
        for (Compartible x : lista) {
            x.compartir();
        }
    }
}
```

Salida:

```
Soy Foto del atardecer
Foto del atardecer se muestra en pantalla completa
Soy Video de la fiesta
Video de la fiesta se reproduce en bucle
Foto del atardecer se comparte con 120 personas
Perfil de Ana se comparte con 300 seguidores
```

10.11 Arquitectura y construcción

Nuestro hilo conductor. Una vivienda y una torre son edificios; una vivienda y un puente se certifican, aunque un puente no es un edificio. (En el código la casa se llama `Vivienda`.)

Clase (plano)	`Casa` -> objeto: la casa de Los Olmos 123
Encapsulamiento	un edificio no supera la altura máxima del plano regulador
Abstracta (es un)	`Edificio` -> `Vivienda`, `Torre`
Interfaz (sabe hacer)	`Certificable` -> `Vivienda` **y** `Puente`
Composición (tiene un)	un `Edificio` tiene `Pisos`; un `Piso` tiene `Habitaciones`

```
// Arquitectura.java
abstract class Edificio {
    protected String nombre;

    Edificio(String nombre) {
        this.nombre = nombre;
    }

    void presentar() {                  // ya esta hecho: lo heredan todos
        System.out.println("Soy " + nombre);
    }

    abstract void construir();                // pendiente: cada hijo lo hace a su
manera
}

interface Certificable {
    void certificar();                        // solo la promesa, sin cuerpo
}

class Vivienda extends Edificio implements Certificable {
    Vivienda(String nombre) {
        super(nombre);
    }

    void construir() {
        System.out.println(nombre + " levanta muros y techo de dos aguas");
    }

    public void certificar() {
        System.out.println(nombre + " recibe el certificado de recepcion final");
    }
}

class Torre extends Edificio {
    Torre(String nombre) {
        super(nombre);
    }

    void construir() {
        System.out.println(nombre + " levanta 40 pisos con grua");
    }
```

```
}

// Puente NO es un Edificio, pero tambien cumple el contrato Certificable
class Puente implements Certificable {
    private String nombre;

    Puente(String nombre) {
        this.nombre = nombre;
    }

    public void certificar() {
        System.out.println(nombre + " recibe el certificado de ingenieria");
    }
}

public class Arquitectura {
    public static void main(String[] args) {
        // new Edificio("x");   // ERROR: Edificio es abstracta, no se puede
instanciar

        Edificio[] grupo = { new Vivienda("Casa de Maipu"), new Torre("Torre
Central") };
        for (Edificio p : grupo) {
            p.presentar();
            p.construir();
        }

        Certificable[] lista = { new Vivienda("Casa de Maipu"), new Puente("Puente
Bicentenario") };
        for (Certificable x : lista) {
            x.certificar();
        }
    }
}
```

Salida:

```
Soy Casa de Maipu
Casa de Maipu levanta muros y techo de dos aguas
Soy Torre Central
Torre Central levanta 40 pisos con grua
Casa de Maipu recibe el certificado de recepcion final
Puente Bicentenario recibe el certificado de ingenieria
```

Parte 11 - Ejercicios para pensar (sin código)

Estos ejercicios **no se programan**. Se hacen en papel, en el cuaderno o en una pizarra. Diseñar bien es la parte difícil; la sintaxis se aprende después.

Formato de respuesta sugerido: una lista de clases, y para cada una sus *atributos* y *métodos*. Si hay relaciones, anótalas como "A **es un** B" o "A **tiene un** B".

Ejercicio 1 - La farmacia

Una farmacia vende medicamentos, atiende a clientes y emite boletas.

1. Escribe las **5 clases** más importantes.
2. A cada una, ponle 3 atributos y 2 métodos.
3. Elige **una regla** que alguna clase debería proteger (encapsulamiento). *Pista: ¿puedes vender un medicamento con receta sin receta?*

Ejercicio 2 - ¿"Es un" o "tiene un"?

Para cada par, decide si es **herencia** o **composición**, y justifica con una frase:

Par	¿Es un / Tiene un?
`Biblioteca` y `Libro`	
`Estudiante` y `Persona`	
`Pizza` y `Ingrediente`	
`Ambulancia` y `Vehiculo`	
`Pelicula` y `Actor`	
`Cuadrado` y `Rectangulo`	

▼ **Respuestas**

Par	Respuesta
`Biblioteca` y `Libro`	**Tiene un**: la biblioteca tiene libros
`Estudiante` y `Persona`	**Es un**: el estudiante es una persona
`Pizza` y `Ingrediente`	**Tiene un**: la pizza tiene ingredientes
`Ambulancia` y `Vehiculo`	**Es un**: la ambulancia es un vehículo
`Pelicula` y `Actor`	**Tiene un**: la película tiene actores (el actor no es una película)
`Cuadrado` y `Rectangulo`	**Es un** en matemáticas, pero cuidado: en código trae problemas (si cambias el ancho de un cuadrado, ¿cambia el alto?). Buen tema para discutir.

Ejercicio 3 - ¿Clase abstracta o interfaz?

Para cada situación, decide qué usarías y por qué:

1. Quieres que `Perro`, `Gato` y `Loro` compartan `nombre` y `edad`, y que cada uno tenga su propio sonido.
2. Quieres que `Foto`, `Perfil` y `Evento` se puedan compartir en una red social. No tienen nada en común salvo eso.
3. Quieres que `Camion`, `Moto` y `Bicicleta` compartan `patente` y un método `detener()` ya hecho, pero que cada uno calcule su propio costo por kilómetro.
4. Quieres que `Tarjeta`, `Transferencia` y `Efectivo` se puedan usar para pagar.
5. Quieres que `Guitarra` y `Violin` tengan `cuerdas`, y que cada uno suene distinto.
6. Quieres que un `Microfono` y una `Guitarra` electrica se puedan conectar a un amplificador.

▼ **Respuestas**

1. **Abstracta** (`Animal`): comparten datos y cada uno completa el sonido.
2. **Interfaz** (`Compartible`): no son de la misma familia.
3. **Abstracta** (`Vehiculo`): comparten dato y método hecho; el costo es el método pendiente.
4. **Interfaz** (`MedioDePago`): son cosas distintas con la misma habilidad.
5. **Abstracta** (`Instrumento`): comparten dato (`cuerdas`).
6. **Interfaz** (`Amplificable`): un micrófono no es un instrumento, pero ambos se conectan al amplificador.

Ejercicio 4 - Tu aplicación favorita

Elige una app que uses todos los días (Spotify, Instagram, Uber, Netflix, WhatsApp...). Haz una lista de:

1. **8 clases** que crees que tiene por dentro.
2. **Una clase abstracta** (algo que existe solo en sus variantes).
3. **Una interfaz** (una habilidad que cruce varias clases).
4. **Dos relaciones "tiene un"**.

Ejercicio 5 - Encuentra la regla

Para cada clase, escribe **una regla** que debería proteger el encapsulamiento:

Clase	Regla
`Ascensor`	
`Cuenta`	
`Termometro`	
`Entrada` (de cine)	
`Semaforo`	

▼ **Ejemplos de respuesta**

- `Ascensor`: no puede subir con la puerta abierta; no supera el último piso.
- `Cuenta`: el saldo no queda negativo sin autorización.
- `Termometro`: no mide bajo el cero absoluto.
- `Entrada`: no se puede usar dos veces.
- `Semaforo`: pasa en orden (verde → amarillo → rojo), y nunca dos luces a la vez.

Ejercicio 6 - Arregla el diseño

Alguien diseñó esto. Encuentra qué está mal y propón una solución.

```
Ave
 ├─ volar()
 ├─ ponerHuevos()

Gorrion  extends Ave
Pinguino extends Ave       <- ¡hereda volar()!
Avion    extends Ave       <- ??
```

Dos problemas. ¿Cuáles? ¿Cómo lo reordenas usando una **interfaz**?

<details>
<summary>▼ Respuesta</summary>

1. El pingüino hereda `volar()`, y no vuela.
2. El avión **no es un ave**: heredó de `Ave` solo para poder volar.

Solución: `Ave` conserva `ponerHuevos()`. Se crea la interfaz `Volador` con `volar()`, que firman `Gorrion` y `Avion` (y quien más vuele). `Pinguino` solo extiende `Ave`, sin firmar `Volador`.
</details>

Ejercicio 7 - Traduce de un área a otra

La estructura de la Parte 10 sirve para cualquier tema. Toma una de las fichas y **tradúcela a un área nueva**, por ejemplo una **biblioteca**:

1. ¿Cuál es la clase abstracta? (*Pista: algo que existe solo en variantes, como* `Material`.)
2. ¿Cuáles son los dos hijos? (*libro, revista...*)
3. ¿Qué interfaz cruza familias? (*algo que se pueda* `prestar()`*: un libro, pero también una sala de estudio o un notebook.*)

Ejercicio 8 - Explícaselo a alguien

Sin usar las palabras *clase*, *objeto* ni *interfaz*, explica en 4 o 5 frases a un familiar **para qué sirve** organizar un programa así. Si te entiende, tú también lo entendiste.

Parte 12 - Autoevaluación

Responde **sin mirar**, y después revisa:

1. ¿Qué diferencia hay entre una clase y un objeto?

2. Si un objeto tiene "estado propio", ¿qué significa eso?
3. ¿Para qué sirve esconder un atributo (`private`)?
4. ¿Qué frase usas para decidir si va herencia? ¿Y composición?
5. ¿Qué significa que una clase sea abstracta?
6. ¿Puedo hacer `new` de una clase abstracta? ¿Y de una interfaz?
7. ¿Cuántas clases puedo heredar? ¿Y cuántas interfaces puedo firmar?
8. ¿Qué es lo que **no** puede hacer un `Pinguino` si lo hacemos extender `Ave` con `volar()`, y cómo se arregla?

<details>
<summary>▼ Respuestas</summary>

1. La clase es el **plano** (la descripción); el objeto es una **cosa concreta** creada a partir de ella.
2. Que sus datos son suyos: cambiar los de un objeto no cambia los de otro.
3. Para que nadie lo deje en un estado inválido; los cambios pasan por métodos que validan.
4. Herencia: "**es un**". Composición: "**tiene un**".
5. Que es un molde: no se puede instanciar, y puede dejar métodos pendientes (abstractos) que cada hijo debe completar.
6. No, a ninguna de las dos.
7. Una sola clase (`extends`); varias interfaces (`implements`).
8. El pingüino heredaría un `volar()` que no puede cumplir. Se arregla sacando `volar()` de `Ave` y poniéndolo en una interfaz `Volador` que firman solo los que vuelan.

</details>

Mapa: de esta guía a las demos

Concepto de esta guía	Dónde se practica
Clase y objeto (Parte 2)	`01-clases-objetos/`
Encapsulamiento (Parte 3)	`02-encapsulamiento/`
Herencia (Parte 4)	`03-herencia/`
Polimorfismo y clase abstracta (Partes 5 y 7)	`04-polimorfismo/`
Composición (Parte 6)	`05-composicion-estaticos/`
Interfaces (Partes 8 y 9)	`07-interfaces/`
Todo junto, en grande	`08-videojuego/` y `09-taller-grupal/`

Para ejecutar los ejemplos

Cada archivo es independiente, y los de `ejemplos/java/` conviven sin chocar de nombres:

```
cd ejemplos/java
javac Medicina.java
java Medicina
```

Son 17 programas: 6 conceptuales y 11 de dominio.